

Tough Trigger Scenario: Revenue Rollup and Tiering Engine

Business Requirement

Your company sells to **Accounts** through **Opportunities** with **Opportunity Products** (OpportunityLineItem). Build trigger logic to maintain these fields:

On Account:

- `Lifetime_Won_Revenue__c`: sum of line item totals across all **Closed Won** Opportunities for that Account **and all its child accounts** (hierarchy up to 5 levels)
- `Customer_Tier__c`: Bronze (< 100K), Silver (100K to 500K), Gold (> 500K)
- `Last_Tier_Change__c`: datetime, updated only when the tier actually changes

On Opportunity:

- `Discount_Exposure__c`: the sum of line item discounts, which must never exceed 15% of Opportunity Amount

Rules and Edge Cases

1. **Line items can change independently.** Insert, update (quantity, price, discount), delete and undelete on OpportunityLineItem must all recalculate the Account totals.
2. **Opportunity stage changes.** Moving into or out of Closed Won must add or subtract revenue. Moving between non-won stages must do nothing and fire **zero** extra DML or queries.
3. **Account reparenting.** If an Opportunity moves from Account A to Account B, A must lose the revenue and B must gain it, along with both of their parent hierarchies.
4. **Account hierarchy reparenting.** If Account X is moved under a new parent, the old and new ancestor chains must be recalculated.
5. **Block bad data.** Reject Closed Won if `Discount_Exposure__c` exceeds 15%, using `addError` on the specific records only, so one bad record in a batch of 200 doesn't fail the others (partial success via `Database.insert(records, false)`).
6. **Tier notifications.** When an Account reaches Gold, create a Task for the Account Owner. Create exactly one, even if the trigger fires multiple times in one transaction.
7. **Concurrency.** Two users closing Opportunities on the same Account at the same moment must not produce a wrong total (think `UNABLE_TO_LOCK_ROW`).

Constraints

- Must handle **200 Opportunities and 10,000 line items** in one transaction (Data Loader, then a bulk API test).
- No SOQL or DML in loops. Stay inside governor limits.
- Must work when called from a **Flow, a batch job and an API** in the same transaction (re-entry and recursion).
- Only one trigger per object, with a handler class. Logic must be unit-testable.
- If total recalculation touches more than a threshold of Accounts (say 500), push the work to **Queueable/Batch** while keeping data consistent.

What Makes This Hard

Trap	Why it bites
Recursion	Account update, then Account trigger, then Opportunity update, then loop. A static boolean isn't enough. Use a static Set<Id> or map of processed state.
Hierarchy rollup	Needs iterative parent traversal with bulk-safe queries, not one query per level per record.
Reparenting	You need Trigger .oldMap vs Trigger .newMap comparison, and both old and new Accounts must be recalculated.
Undelete	after undelete on line items is easy to forget.
Idempotency	Recomputing from source (aggregate SOQL) is safer than incrementing and decrementing deltas.
Row locks	Use FOR UPDATE selectively or retry via Queueable with backoff.
Mixed order of execution	Flows, validation rules and workflow field updates can re-fire your trigger.

Test Requirements

- Bulk test with 200 Opportunities across 50 Accounts in a 3-level hierarchy
- Test for each of the 7 rules above
- A test proving **no more than N SOQL queries** regardless of batch size, using Limits.getQueries() assertions
- Negative test: a mixed batch where 5 records fail validation and 195 succeed

Bonus Challenges

- Make it **metadata-driven** (Custom Metadata Type holds the tier thresholds).
- Add a **bypass switch** (Custom Permission or Hierarchy Custom Setting) so data migrations can skip the logic.
- Explain whether this should have been a **Roll-Up Summary field**, a **Flow**, or **DLRS** instead, and justify why a trigger is or isn't right here. Roll-ups don't work across a lookup relationship like the Account hierarchy, which is one reason to choose a trigger.